



SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 - CONCEPT MAPPING



Course Code:	CSA1216	Course Title:	Computer Architecture
CO Assessed:	C01	Assessment:	ASSESSMENT TOOL 2- CONCEPT MAPPING
Weightage:	35%	Total Marks:	25
C01 Statement:	Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)		
Student Name:	K.Vijay Bhaskar Reddy	Reg.No.:	192425155
Department:	AI & ML	Date:	15/07/2026

Instruction to Students

1. Each question carries 5 mark.
2. Only the appropriate Tools can be used
3. Time allotted for the exam is 60 min

Code of Conduct:

I K.Vijay Bhaskar Reddy(192425155) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

K. Vijay Bhaskar Reddy

ANNEXURE A

1. Construct a detailed concept map showing the relationship between CPU (ALU, Control Unit, Registers), memory (cache, RAM), and I/O devices. Extend the map to include single-bus, multi-bus, and hierarchical bus structures, and clearly illustrate how data and control signals flow among these components during execution. Indicate possible points of delay or bottlenecks and how different bus structures help in improving performance.
2. Develop a comprehensive concept map linking the instruction execution cycle (fetch, decode, execute) with CPU performance parameters such as CPI, clock cycles, and execution time. Show how each stage contributes to overall execution time, and include connections to factors like memory access, instruction complexity, and pipeline stalls. Also, represent how benchmarking is used to evaluate performance.
3. Create a concept map comparing 8085 and 8086 architectures, including their instruction sets, register organization, addressing modes, and memory segmentation (in 8086). Clearly show how these architectural features influence program execution, multitasking capability, and efficiency in real-time applications.
4. Draw a detailed concept map that connects various addressing modes (immediate, direct, register, indirect, indexed) with instruction types (data transfer, arithmetic, control instructions). Illustrate how each addressing mode affects memory access time, instruction size, and execution speed, and show real-world scenarios where specific addressing modes are preferred.
5. Prepare an elaborate concept map integrating number systems (binary and hexadecimal) with instruction representation, memory storage, and debugging processes. Show how data is converted between formats, how instructions are stored and interpreted by the CPU, and why hexadecimal representation is commonly used in low-level programming and system design.

1. CPU, Memory, I/O Devices and Bus Structures

Central Topic

Computer System Organization

Main Concepts

- CPU
- Memory
- I/O Devices
- Bus Structures

Sub-concepts

CPU

- Arithmetic Logic Unit (ALU)
- Control Unit (CU)
- Registers

Memory

- Cache Memory
- RAM (Main Memory)

I/O Devices

- Input Devices (Keyboard, Mouse, Scanner)
- Output Devices (Monitor, Printer)

Bus Structures

- Single Bus
- Multi Bus
- Hierarchical Bus

Relationships (Linking Words)

- CPU **contains** ALU, Control Unit, and Registers.
- ALU **performs** arithmetic and logical operations.
- Control Unit **controls** instruction execution.
- Registers **store** temporary data and instructions.
- Cache **stores** frequently used data.
- RAM **stores** programs and data.
- Bus **transfers** data, address, and control signals.
- Single Bus **uses** one communication path.
- Multi Bus **provides** parallel communication.
- Hierarchical Bus **improves** scalability and performance.
- I/O Devices **exchange** data with the CPU through buses.

Hierarchy (General → Specific)

Computer System Organization → CPU, Memory, I/O Devices → ALU, Control Unit, Registers, Cache, RAM, Input Devices, Output Devices → Bus Structures → Single Bus, Multi Bus, Hierarchical Bus.

Cross-links

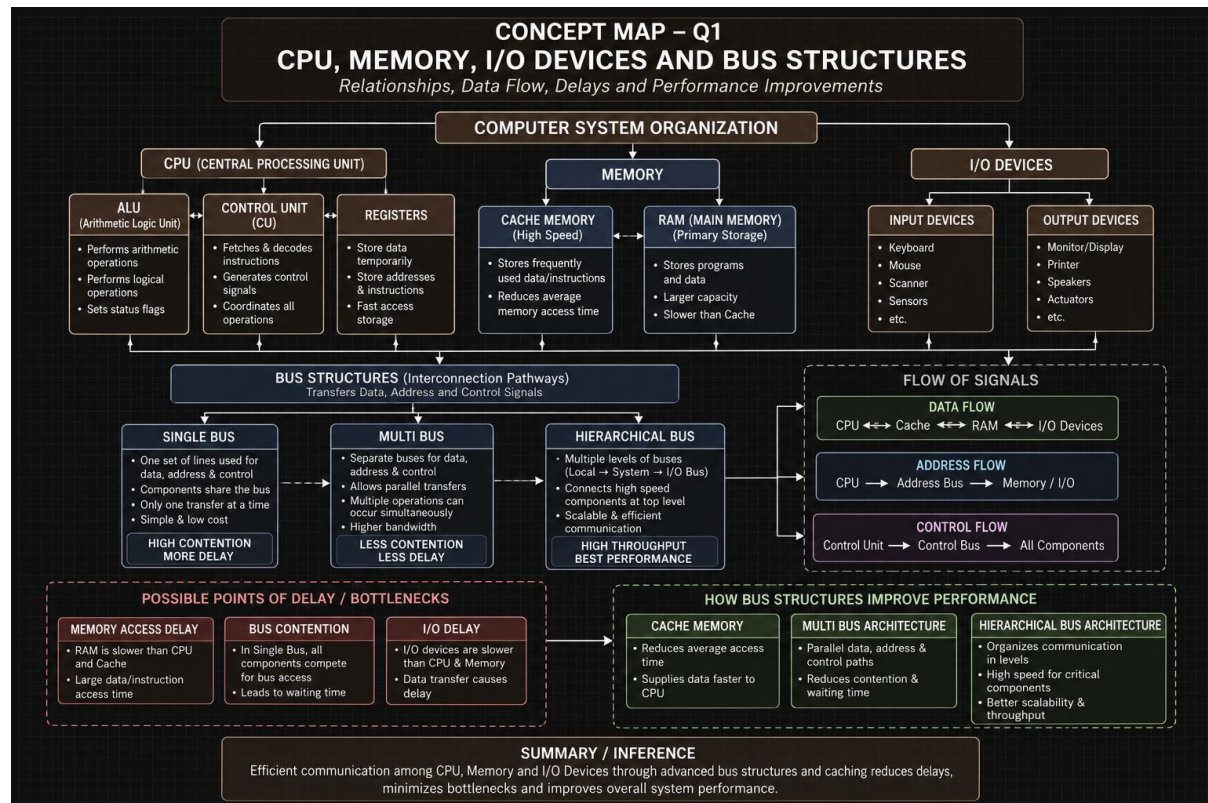
- Cache **reduces** memory access time.
- Registers **communicate** with ALU during execution.
- Bus Structures **affect** data transfer speed.
- Multi Bus and Hierarchical Bus **reduce** bottlenecks.

Real-world Application

Computers, laptops, smartphones, servers, and embedded systems.

Summary/Inference

Efficient communication between CPU, memory, and I/O devices through advanced bus structures improves system performance and minimizes delays.



2. Instruction Execution Cycle and CPU Performance

Central Topic

Instruction Execution Cycle

Main Concepts

- Fetch
- Decode
- Execute
- CPU Performance

Sub-concepts

Fetch

- Read instruction from memory
- Load instruction into Instruction Register

Decode

- Identify opcode
- Determine operands

Execute

- Perform operation
- Store result

CPU Performance

- CPI (Cycles Per Instruction)
- Clock Cycles
- Execution Time
- Benchmarking

Relationships (Linking Words)

- Fetch **reads** instructions from memory.
- Decode **interprets** the instruction.
- Execute **performs** the required operation.
- Memory Access **affects** fetch time.
- Instruction Complexity **influences** execution time.
- Pipeline Stalls **increase** CPI.
- Clock Cycles **determine** execution speed.
- Benchmarking **evaluates** processor performance.

Hierarchy (General → Specific)

Instruction Execution Cycle → Fetch, Decode, Execute → CPU Performance → CPI, Clock Cycles, Execution Time, Benchmarking.

Cross-links

- Memory access **increases** execution time.
- Pipeline stalls **increase** CPI.
- Benchmarking **compares** processor performance.
- Clock frequency **reduces** execution time.

Real-world Application

Performance comparison of Intel, AMD, and ARM processors.

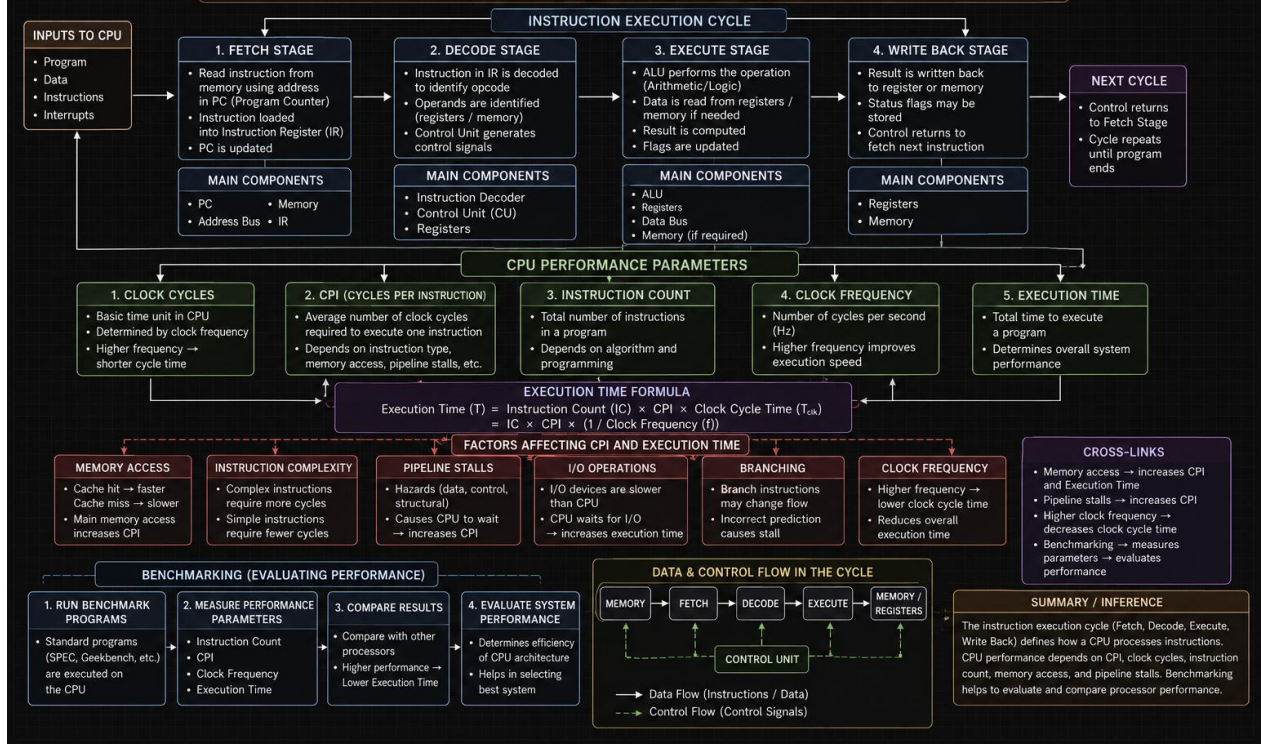
Summary/Inference

The efficiency of the fetch, decode, and execute stages directly determines CPU performance and execution time.

CONCEPT MAP – Q2

INSTRUCTION EXECUTION CYCLE AND CPU PERFORMANCE

Linking Execution Stages with Performance Parameters



3. Comparison of 8085 and 8086 Architectures

Central Topic

8085 vs 8086 Architecture

Main Concepts

- Instruction Set
- Register Organization
- Addressing Modes
- Memory Segmentation

Sub-concepts

8085

- 8-bit processor
- 64 KB memory
- Six general-purpose registers
- Limited addressing modes

8086

- 16-bit processor
- 1 MB memory
- General-purpose and segment registers
- Memory Segmentation
- Advanced addressing modes

Relationships (Linking Words)

- 8086 **supports** larger memory than 8085.
- Memory Segmentation **improves** memory management.
- Registers **increase** execution efficiency.
- Advanced Addressing Modes **improve** flexibility.
- Instruction Set **determines** processing capability.
- Segment Registers **manage** code, data, stack, and extra memory.

Hierarchy (General → Specific)

Microprocessor Architecture → 8085 and 8086 → Registers, Instruction Set, Addressing Modes, Memory Segmentation.

Cross-links

- Memory Segmentation **supports** multitasking.
- More Registers **increase** execution speed.
- Advanced Instructions **improve** real-time performance.

Real-world Application

Industrial automation, embedded systems, robotics, and control systems.

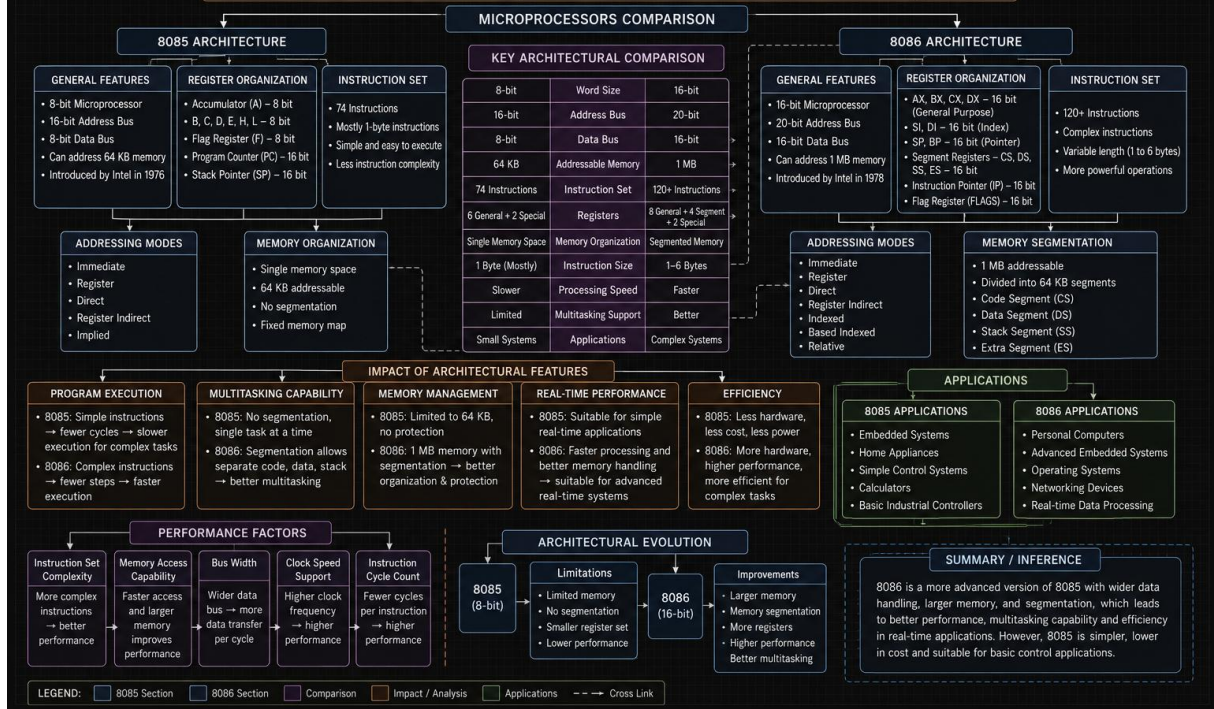
Summary/Inference

The 8086 architecture provides higher speed, larger memory, better multitasking, and greater efficiency than the 8085 architecture.

CONCEPT MAP – Q3

COMPARISON OF 8085 AND 8086 ARCHITECTURES

Architecture Differences and Their Impact on Performance, Multitasking and Real-time Applications



4. Addressing Modes and Instruction Types

Central Topic

Addressing Modes

Main Concepts

- Immediate Addressing
- Direct Addressing
- Register Addressing
- Indirect Addressing
- Indexed Addressing

Sub-concepts

Instruction Types

- Data Transfer Instructions
- Arithmetic Instructions
- Control Instructions

Performance Factors

- Memory Access Time
- Instruction Size
- Execution Speed

Relationships (Linking Words)

- Immediate Addressing **stores** data within the instruction.
- Direct Addressing **accesses** memory directly.
- Register Addressing **uses** CPU registers.
- Indirect Addressing **references** memory through pointers.
- Indexed Addressing **supports** array processing.
- Addressing Modes **affect** memory access time.
- Instruction Types **use** different addressing modes.

Hierarchy (General → Specific)

Addressing Modes → Immediate, Direct, Register, Indirect, Indexed → Data Transfer, Arithmetic, Control Instructions → Performance Factors.

Cross-links

- Register Addressing **provides** fastest execution.
- Indexed Addressing **supports** arrays.
- Indirect Addressing **supports** linked data structures.
- Memory Access Time **affects** execution speed.

Real-world Application

Array processing, database indexing, pointer operations, and embedded programming.

Summary/Inference

Selecting the appropriate addressing mode improves program efficiency, reduces execution time, and optimizes memory usage.

CONCEPT MAP - Q4

ADDRESSING MODES AND INSTRUCTION TYPES

Relationship with Memory Access, Instruction Size, Execution Speed and Real-World Applications

ADDRESSING MODES (Ways to Specify Operands)

1. IMMEDIATE ADDRESSING

- Operand is part of the instruction itself
- No memory access for operand

Example: `MOV A, #55H`
Use: Constants, small data

2. DIRECT ADDRESSING

- Address of operand is given in the instruction
- Operand is in memory location

Example: `MOV A, 2050H`
Use: Global variables

3. REGISTER ADDRESSING

- Operand is in a register
- Fastest access

Example: `MOV A, B`
Use: High speed operations, temporary data

4. INDIRECT ADDRESSING

- Instruction contains address of memory location
- That location contains the operand

Example: `MOV A, @2000H`
Use: Pointers, dynamic data

5. INDEXED ADDRESSING

- Effective address = Base + Index
- Useful for arrays and tables

Example: `MOV A, TABLE[SI]`
Use: Array access, tables

USED BY INSTRUCTION TYPES

A. DATA TRANSFER INSTRUCTIONS

Examples: `MOV, LDA, STA, XCHG`
Purpose: Move data between registers and memory

B. ARITHMETIC INSTRUCTIONS

Examples: `ADD, SUB, MUL, DIV, INC, DEC`
Purpose: Perform arithmetic operations on data

C. CONTROL INSTRUCTIONS

Examples: `JMP, CALL, RET, JZ, JNZ`
Purpose: Control the flow of program execution

CROSS-LINKS / RELATIONSHIPS

- Register addressing → fastest execution
- Immediate addressing → no memory access
- Direct & Indirect → require memory access
- Indexed addressing → best for arrays
- Addressing mode affects → time, size, speed

EFFECT ON PERFORMANCE

Addressing Mode	Memory Access	Instruction Size	Execution Speed
Immediate	None	Small	Very Fast
Register	None	Small	Fastest
Direct	One	Medium	Medium
Indirect	Two	Medium	Slower
Indexed	One (+ Index Calc.)	Large	Medium to Fast

REAL-WORLD APPLICATIONS

123	Immediate	→ Used for constants, flags, configuration values.
Ⓡ	Register	→ Used in CPU operations, counters, accumulators.
🏠	Direct	→ Used for accessing global variables, I/O mapped memory.
🔗	Indirect	→ Used in linked lists, dynamic memory, pointer operations.
📄	Indexed	→ Used in array processing, tables, string handling.

SUMMARY / INFERENCE

Different addressing modes provide different ways to access operands. The choice of addressing mode affects memory access time, instruction size and execution speed. Register and Immediate addressing are fastest, while Indirect is slower due to extra memory access. Indexed addressing is very useful for arrays and tables. Selecting the right addressing mode helps in writing efficient, fast and compact programs.

FLOW OF EXECUTION (USING ADDRESSING MODES)



LEGEND

- Hierarchy / Main Flow
- - - Relationship / Usage
- 📦 Addressing Mode
- 📄 Instruction Type
- 📊 Performance Table
- 📌 Applications
- 🔗 Cross Links
- 📋 Summary

5. Number Systems, Memory Storage and Debugging

Central Topic

Number Systems

Main Concepts

- Binary Number System
- Hexadecimal Number System
- Instruction Representation
- Memory Storage
- Debugging

Sub-concepts

Binary

- Machine Language
- Bit Representation

Hexadecimal

- Compact Representation
- Memory Address Representation

Memory

- Binary Storage
- Address Allocation

CPU

- Instruction Fetch
- Instruction Decode
- Instruction Execution

Debugging

- Memory Dump
- Machine Code Analysis

Relationships (Linking Words)

- Binary **represents** machine instructions.
- Hexadecimal **simplifies** binary representation.
- Memory **stores** binary data.
- CPU **interprets** binary instructions.
- Debugging Tools **display** hexadecimal values.
- Binary **converts** to hexadecimal for readability.

Hierarchy (General → Specific)

Number Systems → Binary and Hexadecimal → Instruction Representation → Memory Storage → CPU Execution → Debugging.

Cross-links

- Binary Data **is converted into** hexadecimal.
- Memory Addresses **are represented by** hexadecimal values.
- CPU **executes** binary instructions.
- Debuggers **display** memory in hexadecimal format.

Real-world Application

Assembly language programming, embedded systems, operating system development, and hardware debugging.

CONCEPT MAP – Q5

NUMBER SYSTEMS, INSTRUCTION REPRESENTATION, MEMORY STORAGE AND DEBUGGING

Connecting Binary and Hexadecimal to How Computers Store, Process and Debug Data

